

Smart contract audit

rain.vats

Content

Project description	3
Executive summary	3
Reviews	3
Technical analysis and findings	5
Security findings	6
**** Critical	6
*** High	6
H01 - ERC1155 Acceptance Callback Enables Persistent Operator Approval Via _msgSender Spoofing	6
** Medium	7
* Low	7
Informational	7
I01 - Owner Freeze Precedence Over Confiscation Should Be Documented	7
I02 - Generic Receipt Metadata Compatibility For OffchainAssetReceiptVault Should Be Documented	8
General Risks and Assumptions	9
Approach and methodology	10



Project description

This repository implements the Rain VATS (Verifiable Asset Tokenisation System) receipt-vault model for tokenising assets with paired ERC-20 share accounting and ERC-1155 receipt provenance. It provides abstract receipt-vault infrastructure, concrete off-chain-asset and oracle-priced vaults, receipt token implementations, authorizers, oracle adapters, and deployment helpers.

The core system binds fungible ERC-20 vault shares to ERC-1155 receipts minted and burned 1:1 at deposit and withdrawal time. Shares remain suitable for normal DeFi liquidity and transfer flows, while receipts preserve per-mint provenance, audit context, and burn authority for the specific asset or deposit event that justified the shares.

The repository includes an `OffchainAssetReceiptVault` for real-world or externally managed assets, with certification, configurable authorisation, transfer controls, confiscation hooks, and role-based operational permissions. It also includes an `ERC20PriceOracleReceiptVault` that mints shares against ERC-20 deposits using oracle-derived share ratios while preserving receipt-based withdrawal accounting.

Executive summary

Type	Vault
Languages	Solidity
Methods	Architecture Review, Manual Review, Unit Testing, Functional Testing, Automated Review
Documentation	README.md
Repositories	https://github.com/rainlanguage/rain.vats

Reviews

Date	Repository	Commit
22/01/26	rain.vats	15cbe7e270daa75d237a59aa9f46b55f37d30296
22/05/26	rain.vats	cb4d925b42f33eda781b73a21fb5fcc52a1243c2 tag v0.1.5



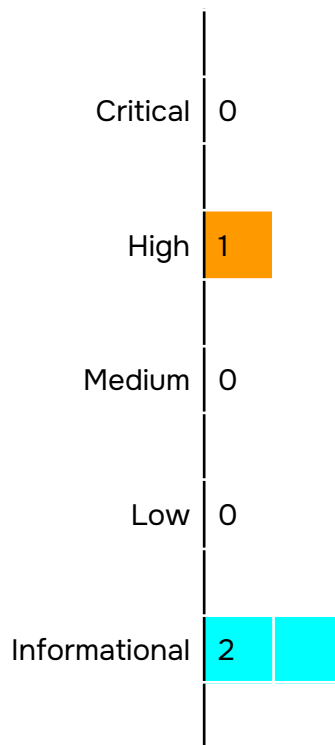
Scope

Contracts and their dependencies

src/abstract/OwnerFreezable.sol
src/abstract/PriceOracleV2.sol
src/abstract/ReceiptVault.sol
src/interface/IPriceOracleV2.sol
src/interface/IReceiptManagerV2.sol
src/interface/IReceiptVaultV3.sol
src/interface/IERC20PriceOracleReceiptVaultCloneDeployerV2.sol
src/interface/IAuthorizableV1.sol
src/interface/IERC20PriceOracleReceiptVaultCloneDeployerV1.sol
src/interface/IFtsoV2LTSFeedOracleV1.sol
src/interface/IPythOracleV2.sol
src/interface/IFtsoV2LTSFeedOracleV2.sol
src/interface/IPythOracleV1.sol
src/interface/IAuthorizeV1.sol
src/interface/ICertifiableV1.sol
src/interface/IOffchainAssetReceiptVaultBeaconSetDeployerV2.sol
src/interface/IOffchainAssetReceiptVaultBeaconSetDeployerV1.sol
src/interface/IReceiptV3.sol
src/interface/IOwnerFreezableV1.sol
src/interface/deprecated/IReceiptManagerV1.sol
src/interface/deprecated/IPriceOracleV1.sol
src/interface/deprecated/IReceiptVaultV2.sol
src/interface/deprecated/IReceiptVaultV1.sol
src/interface/deprecated/IReceiptV2.sol
src/interface/deprecated/IReceiptV1.sol
src/interface/deprecated/IReceiptOwnerV1.sol
src/concrete/deploy/ERC20PriceOracleReceiptVaultCloneDeployer.sol
src/concrete/deploy/OffchainAssetReceiptVaultBeaconSetDeployer.sol
src/concrete/oracle/TwoPriceOracleV2.sol
src/concrete/oracle/SceptreStakedFlrOracle.sol
src/concrete/oracle/PythOracle.sol
src/concrete/oracle/FtsoV2LTSFeedOracle.sol
src/concrete/receipt/ERC20PriceOracleReceipt.sol
src/concrete/receipt/Receipt.sol
src/concrete/authorize/OffchainAssetReceiptVaultAuthorizerV1.sol
src/concrete/authorize/OffchainAssetReceiptVaultPaymentMintAuthorizerV1.sol
src/concrete/vault/OffchainAssetReceiptVault.sol
src/concrete/vault/ERC20PriceOracleReceiptVault.sol
src/error/ErrReceiptVault.sol
src/error/ErrDeployer.sol
src/error/ErrReceipt.sol
src/error/ErrTwoPriceOracleV2.sol
src/vendor/pyth-sdk/PythStructs.sol
src/vendor/pyth-sdk/IPythEvents.sol
src/vendor/pyth-sdk/IPyth.sol



Technical analysis and findings



Security findings

**** Critical

No critical severity issue found.

*** High

H01 - ERC1155 Acceptance Callback Enables Persistent Operator

Approval Via `_msgSender` Spoofing

Receipt uses the `withSender` modifier to write an arbitrary sender value into storage for the full duration of `managerMint()`, `managerBurn()`, and `managerTransferFrom()`. During that period, `Receipt._msgSender()` returns the stored sender instead of the actual caller. ERC1155 mints and transfers invoke receiver acceptance callbacks before returning, allowing an attacker-controlled receiver to re-enter Receipt while the spoofed sender is still active. Because inherited OpenZeppelin ERC1155 functions rely on `_msgSender()`, the receiver can call `setApprovalForAll(attacker, true)` and have the approval recorded as if it came from the spoofed depositor/user rather than from the malicious receiver contract.

Impact: The attacker can obtain persistent operator approval over the spoofed user's ERC1155 receipts across all IDs. After the original mint/transfer finishes, the attacker can use that approval to transfer the victim's existing receipts and any receipts they later acquire. Since receipts are required for withdrawal/redemption flows, this can result in loss of the victim's ability to redeem the corresponding shares/assets. The same root cause can also allow callback-time receipt transfers to be authorised under the spoofed operator.

One of the most probable attack vectors – social engineering via a fake staking integration. A realistic exploitation path relies on deceiving users into voluntarily depositing to a malicious contract as receiver (frontend compromise, by contrast, would enable simpler and more direct fund theft without this mechanism):

1. Attacker deploys `MaliciousStaking`, a contract that implements `IERC1155Receiver` and advertises receipt staking with token bonuses.
2. Users call `vault.deposit/assets, MaliciousStaking, ...)`, believing they are staking receipts into a yield strategy.
3. `ReceiptVault` calls `receipt().managerMint(user, MaliciousStaking, id, shares, ...)`. The `withSender()` modifier sets `s.sender = user`.
4. Inside the ERC1155 mint, OZ calls `MaliciousStaking.onERC1155Received(operator=user, ...)`.
5. `MaliciousStaking` calls `receipt.setApprovalForAll(attacker, true)`. Because `_msgSender()` returns `user`, the approval is recorded as `isApprovedForAll[user][attacker] = true`.



6. After the transaction, the attacker transfers all of the user's receipts – including those from previous deposits – blocking the user from ever redeeming them.

Path: `src/concrete/receipt/Receipt.sol`

Recommendation: Consider reviewing the `withSender/_msgSender()` pattern in the context of ERC1155 acceptance callbacks. One approach is to remove the global `_msgSender()` override and instead pass the depositor's address as an explicit parameter to `_receiptInformation` only, since that is the only function that requires it for event emission. Under this design, inherited ERC1155 functions would observe `msg.sender` during callbacks and would no longer be susceptible to identity spoofing. Any changes to this pattern should be validated against the authorisation flow in `_update()` to confirm that `authorizeReceiptTransfer3()` still receives the correct operator identity.

Found in: `cb4d925b`

Status: New

**** Medium**

No medium severity issue found.

*** Low**

No low severity issue found.

Informational

I01 - Owner Freeze Precedence Over Confiscation Should Be Documented

Confiscation transfers use the normal ERC20/ERC1155 transfer paths and therefore still pass through `OwnerFreezable` checks. As a result, an active owner freeze blocks confiscation unless the relevant sender or receiver has been allowlisted through `ownerFreezeAlwaysAllowFrom` or `ownerFreezeAlwaysAllowTo`. This may be intended behaviour, but the current confiscation, NatSpec says, bypasses transfer restrictions due to system freeze, which can be read as applying to all freeze mechanisms. In practice, confiscation bypasses certification-expiry restrictions through the authorizer role logic, but it does not bypass the owner freeze by default.

Impact: Operators or business owners may misunderstand which emergency actions remain available during the owner freeze. If they expect confiscation or recovery



transfers to work during the owner freeze, those actions can fail unless allowlists are configured. This is primarily a documentation and operational-readiness concern.

Path: src/abstract/OwnerFreezable.sol,
src/concrete/vault/OffchainAssetReceiptVault.sol

Recommendation: Consider clarifying in NatSpec and operational documentation that owner freeze takes precedence over confiscation unless allowlists are configured. If confiscation must remain possible during owner freeze, operators may want to preconfigure `ownerFreezeAlwaysAllowFrom` / `ownerFreezeAlwaysAllowTo` as part of their emergency procedure.

Found in: cb4d925b

Status: New

I02 - Generic Receipt Metadata Compatibility For OffchainAssetReceiptVault Should Be Documented

`OffchainAssetReceiptVault` explicitly requires `receiptVaultConfig.asset == address(0)` because its assets are off-chain. The generic `Receipt` implementation builds metadata by calling `_vaultAssetSymbol()`, which calls `IERC20Metadata(IReceiptVaultV3(manager).asset()).symbol()`. For an off-chain vault, this resolves to `IERC20Metadata(address(0)).symbol()`, which can revert because `address(0)` is not an ERC20 metadata contract. This may be acceptable if off-chain vault deployments are expected to use a dedicated receipt implementation or if metadata support is intentionally handled off-chain.

Impact: ERC1155 metadata lookups for receipts associated with `OffchainAssetReceiptVault` may fail when the generic `Receipt` implementation is used. This does not affect vault accounting or asset safety, but it can affect wallets, marketplaces, explorers, indexers, and provenance display.

Path: src/concrete/receipt/Receipt.sol,
src/concrete/vault/OffchainAssetReceiptVault.sol

Recommendation: Consider documenting in the NatSpec of `OffchainAssetReceiptVault` and `_vaultAssetSymbol()` whether the generic `Receipt` implementation is intended for use with off-chain vaults, and if not, which receipt implementation should be used instead.

Found in: cb4d925b

Status: New



General Risks and Assumptions

- ERC20-backed receipt vaults assume standard ERC20 transfer semantics. The `ReceiptVault` deposit flow does not support fee-on-transfer, deflationary, rebasing-on-transfer, or otherwise non-standard ERC20 assets that credit the vault with less than the requested deposit amount. Developers and business owners should only configure assets whose transfers deliver the exact nominal amount used for share and receipt accounting.
- `ReceiptVault` is an ERC4626-like interface with an additional ERC1155 receipt requirement. Withdrawal and redemption require the owner to hold both ERC20 shares and the corresponding ERC1155 receipt ID. Because these balances can be transferred independently, integrators should not rely only on a receipt balance when deciding whether an account can redeem; both the share balance and the receipt balance must be considered.
- Off-chain assets require trust. The contracts can track receipts and enforce mint/burn accounting, but they cannot prove that real-world assets exist, are properly held, valued, insured, or legally redeemable.
- Off-chain vaults rely on configured roles for minting, certification, authorisation, transfer control, and confiscation. Poor governance or compromised admin keys can directly affect users and token value.
- Some vaults can freeze or limit share and receipt transfers based on certification, authorisation, compliance, or ownership controls. Businesses should disclose this clearly to users and partners.
- For price-oracle vaults, incorrect or stale oracle data can mint the wrong number of shares and damage vault economics. Oracle choice and monitoring are business-critical.



Approach and methodology

To establish a uniform evaluation, we define the following terminology in accordance with the OWASP Risk Rating

Methodology:

	Likelihood Indicates the probability of a specific vulnerability being discovered and exploited in real-world scenarios
	Impact Measures the technical loss and business repercussions resulting from a successful attack
	Severity Reflects the comprehensive magnitude of the risk, combining both the probability of occurrence (likelihood) and the extent of potential consequences (impact)

Likelihood and impact are divided into three levels: High H, Medium M, and Low L. The severity of a risk is a blend of these two factors, leading to its classification into one of four tiers: Critical, High, Medium, or Low.

When we identify an issue, our approach may include deploying contracts on our private testnet for validation through testing. Where necessary, we might also create a Proof of Concept PoC to demonstrate potential exploitability. In particular, we perform the audit according to the following procedure:

	Advanced DeFi Scrutiny We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs
	Semantic Consistency Checks We then manually check the logic of implemented smart contracts and compare with the description in the white paper.
	Security Analysis The process begins with a comprehensive examination of the system to gain a deep understanding of its internal mechanisms, identifying any irregularities and potential weak spots.